Computer Architecture
(EE 371 / CE 321 / CS 330)

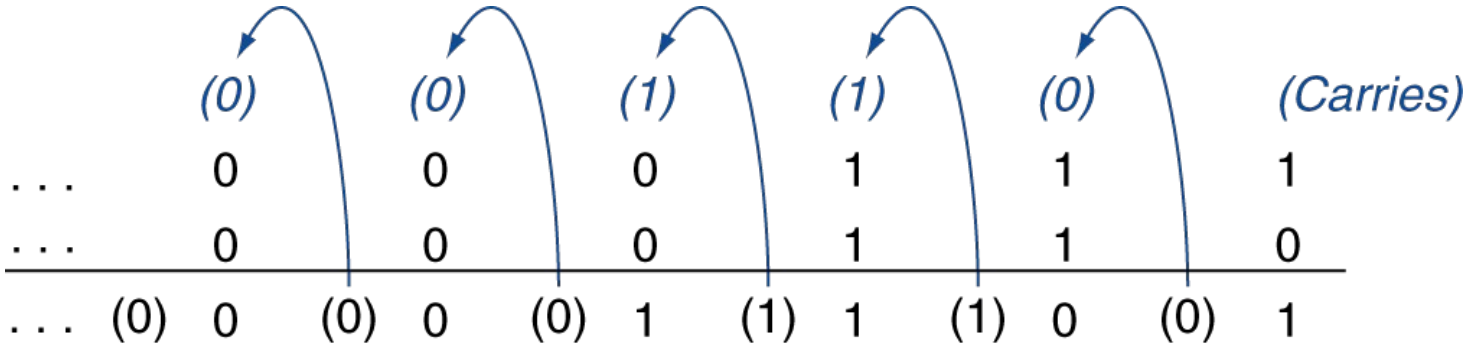
Dr. Farhan Khan
Assistant Professor,
Dhanani School of Science & Engineering,
Habib University

Outline: Unit 3

- Arithmetic for Computers: Review
- Digital Logic Design: Review
- Arithmetic Logic Unit (ALU)
- Single-Cycle Processor Implementation

Integer Addition

$$
\begin{array}{ll}
 & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{two} = 7_{ten} \\
+ & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000110_{two} = 6_{ten} \\
\hline
= & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00001101_{two} = 13_{ten}
\end{array}
$$



Integer Addition: Overflow

$$
\begin{array}{rl}
 & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{two} - 7_{ten} \\
+ & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000110_{two} - 6_{ten} \\
\hline
- & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00001101_{two} - 13_{ten}
\end{array}
$$

Overflow occurs when the result from an operation cannot be represented with the available hardware, in this case a 64-bit word

- Adding +ve and –ve operands
  - Overflow is not possible
- Adding two +ve number
  - Overflow has occurred if the sign bit of the result is 1
- Adding two –ve numbers
  - Overflow has occurred if the sign bit of the result is 0

Integer Subtraction

- Subtraction through Addition
  - Add first operand and negation of the second operand
  - Example: 7 – 6 = 7 + (-6)

$$
\begin{array}{ll}
 & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{two} = 7_{ten} \\
+ & 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111010_{two} = -6_{ten} \\
\hline
= & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000001_{two} = 1_{ten}
\end{array}
$$

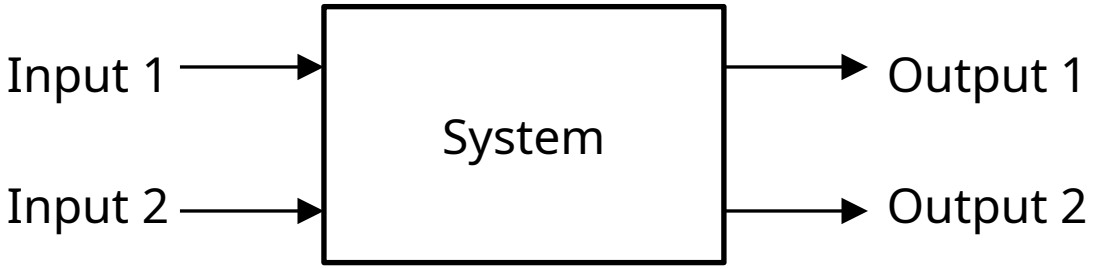
Integer Subtraction: Overflow

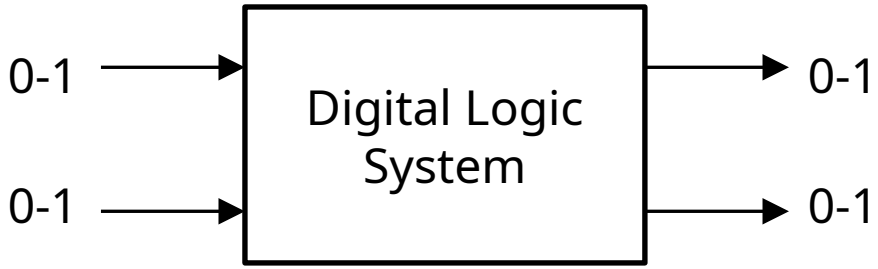
$$
\begin{array}{ll}
 & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{two} - 7_{ten} \\
+ & 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111010_{two} - -6_{ten} \\
\hline
= & 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000001_{two} - 1_{ten}
\end{array}
$$

Overflow occurs when the result from an operation cannot be represented with the available hardware, in this case a 64-bit word

- Subtracting two +ve or two –ve operands
  - Overflow is not possible
- Subtracting +ve from -ve operand
  - Overflow has occurred if the sign bit of the result is 0
- Subtracting –ve from +ve operands
  - Overflow has occurred if the sign bit of the result is 1

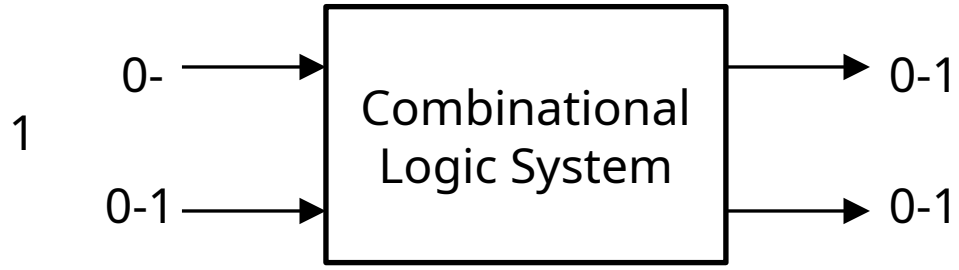
Review of Logic Design



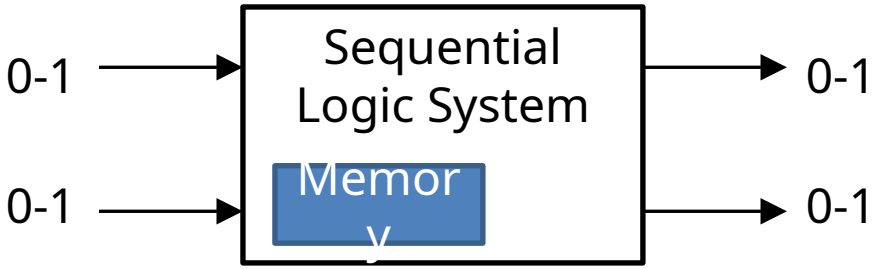


Review of Logic Design

- 2 Types of Logic Systems



The output of a combinational logic system depends only on the current input.



The output of a sequential logic system can depend both on the inputs and internal memory.

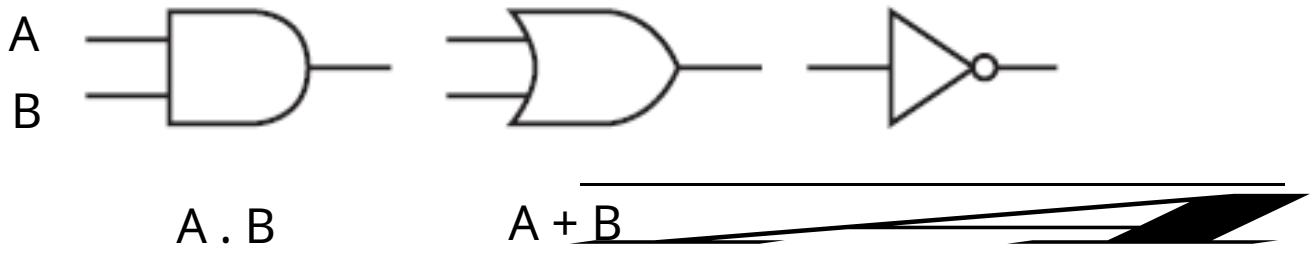
Review of Logic Design

- Truth Table
  - Because a combinational logic block contains no memory, it can be completely specified by defining the values of the outputs for each possible set of input values.
  - Such a description is normally given as a truth table.

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

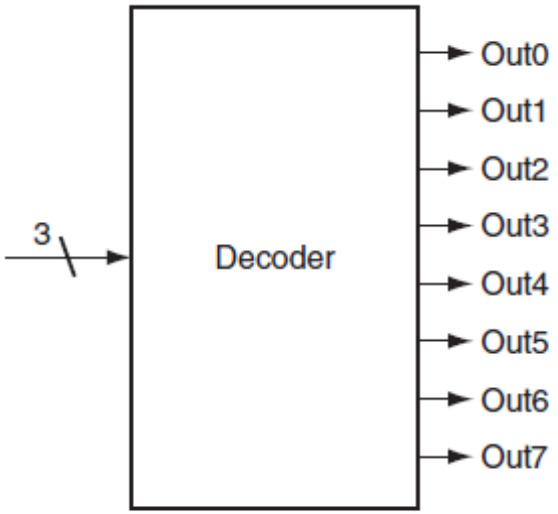
Review of Logic Design

- Logic Gates
  - Logic blocks/systems are built from Logic Gates that implement basic logic functions



Review of Logic Design

- Decoder
  - The decoder has an n-bit input and 2^n outputs, where only one output is asserted for each input combination



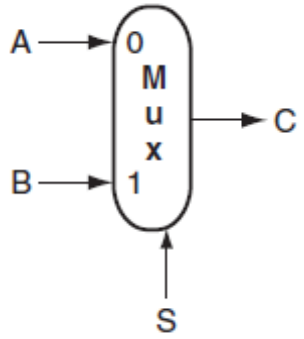
a. A 3-bit decoder

Inputs			Outputs							
12	11	10	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

b. The truth table for a 3-bit decoder

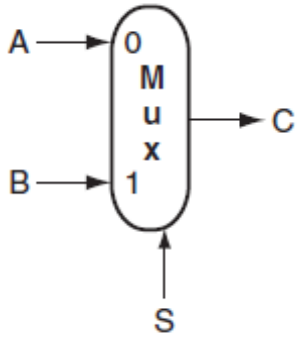
Review of Logic Design

- Multiplexor (Selector)
  - Consider the two-input multiplexor.
  - This multiplexor has three inputs: two data values and a selector (or control) value.
  - The selector value determines which of the inputs becomes the output.



Review of Logic Design

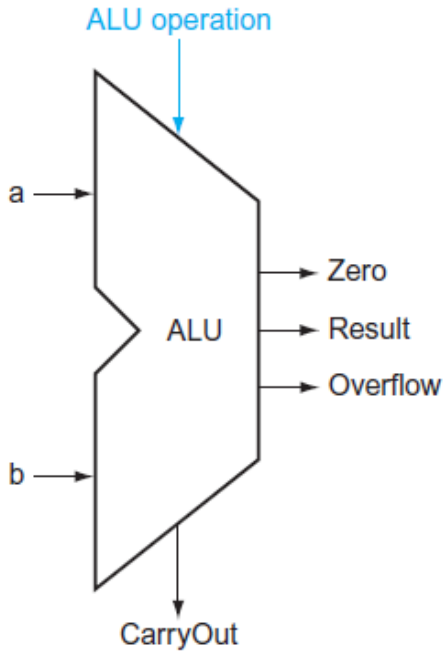
- Multiplexor (Selector)
  - Consider the two-input multiplexor.
  - This multiplexor has three inputs: two data values and a selector (or control) value.
  - The selector value determines which of the inputs becomes the output.



  - Multiplexors can be created with an arbitrary number of data inputs.

Arithmetic Logic Unit (ALU)

- The arithmetic logic unit (ALU) is the brain of the computer, the device that performs the arithmetic operations like addition and subtraction or logical operations like AND and OR.

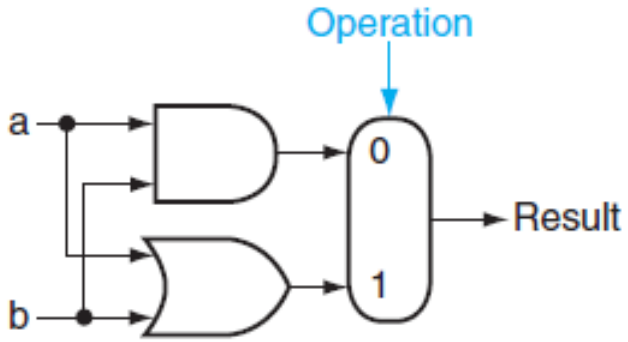


Arithmetic Logic Unit (ALU)

- The arithmetic logic unit (ALU) is the brain of the computer, the device that performs the arithmetic operations like addition and subtraction or logical operations like AND and OR.

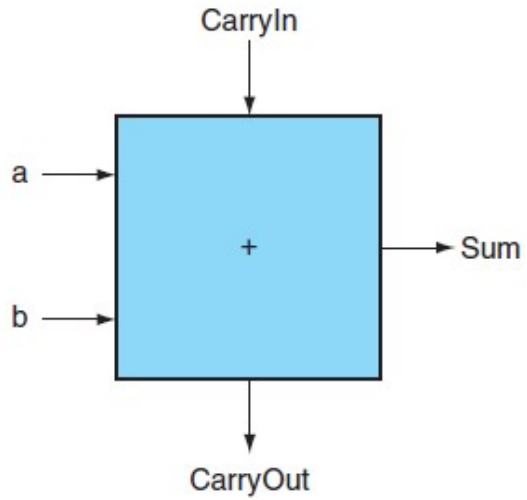
- Let’s focus on 1-bit ALU and progressively build an ALU with more functionality

Arithmetic Logic Unit (ALU)



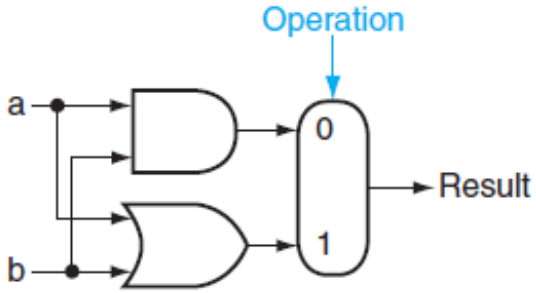
The 1-bit Logical Unit for AND and OR.

1-bit Adder

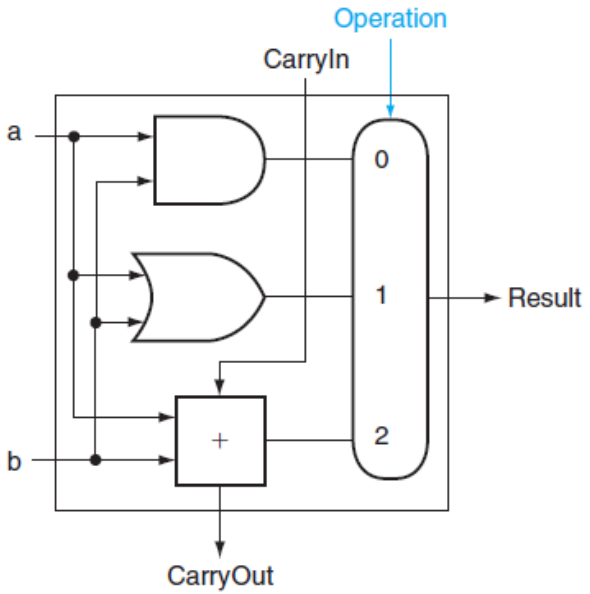


Inputs			Outputs		Comments
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{two}$
0	0	1	0	1	$0 + 0 + 1 = 01_{two}$
0	1	0	0	1	$0 + 1 + 0 = 01_{two}$
0	1	1	1	0	$0 + 1 + 1 = 10_{two}$
1	0	0	0	1	$1 + 0 + 0 = 01_{two}$
1	0	1	1	0	$1 + 0 + 1 = 10_{two}$
1	1	0	1	0	$1 + 1 + 0 = 10_{two}$
1	1	1	1	1	$1 + 1 + 1 = 11_{two}$

Arithmetic Logic Unit (ALU)

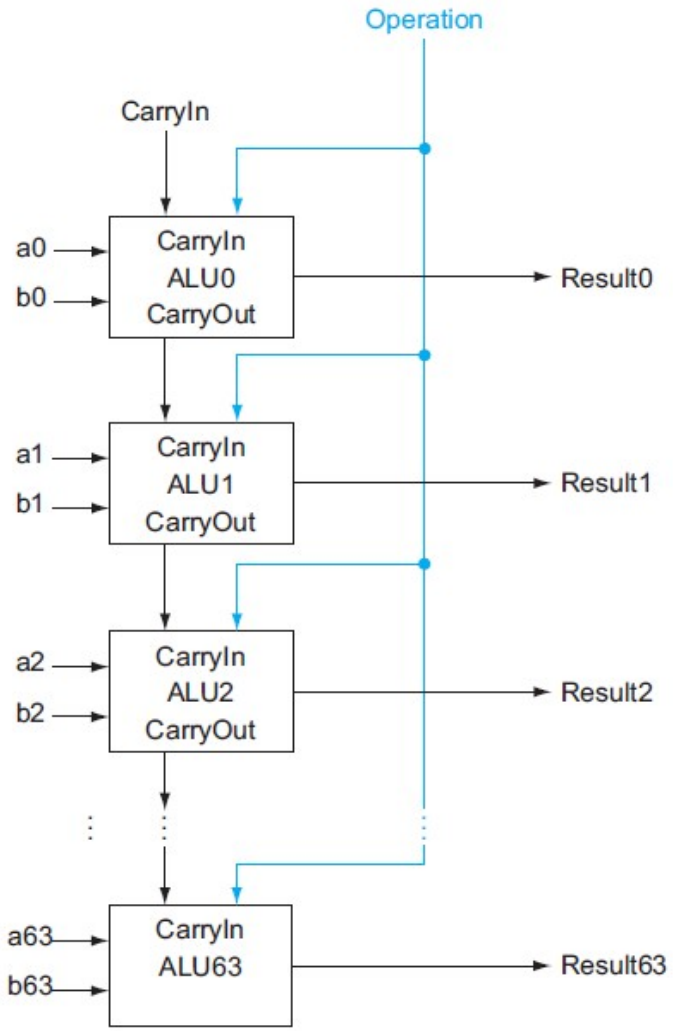


The 1-bit Logical Unit for AND and OR.



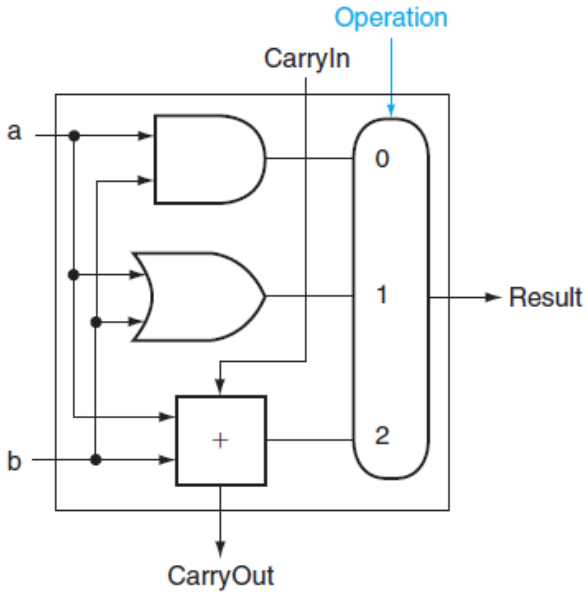
The 1-bit ALU that performs AND, OR, and addition.

Arithmetic Logic Unit (ALU)

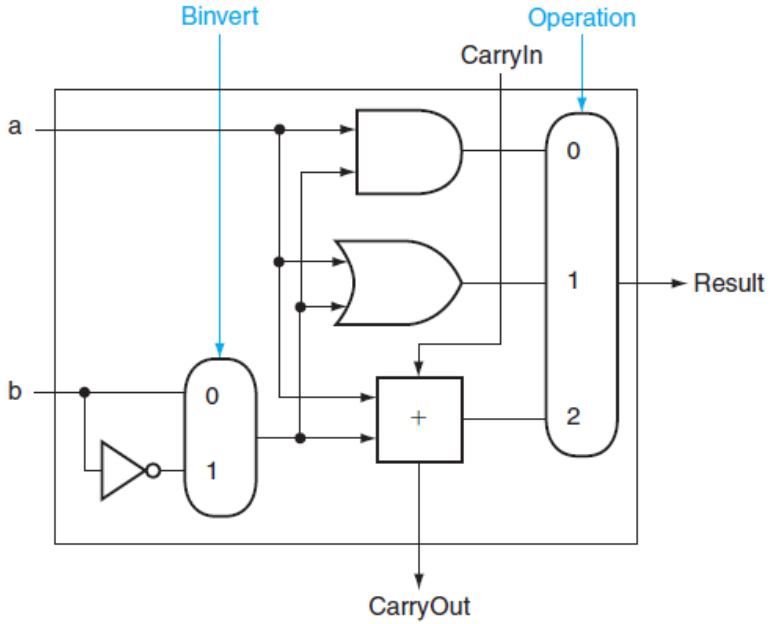


A 64-bit ALU constructed from 64 1-bit ALUs

Arithmetic Logic Unit (ALU)



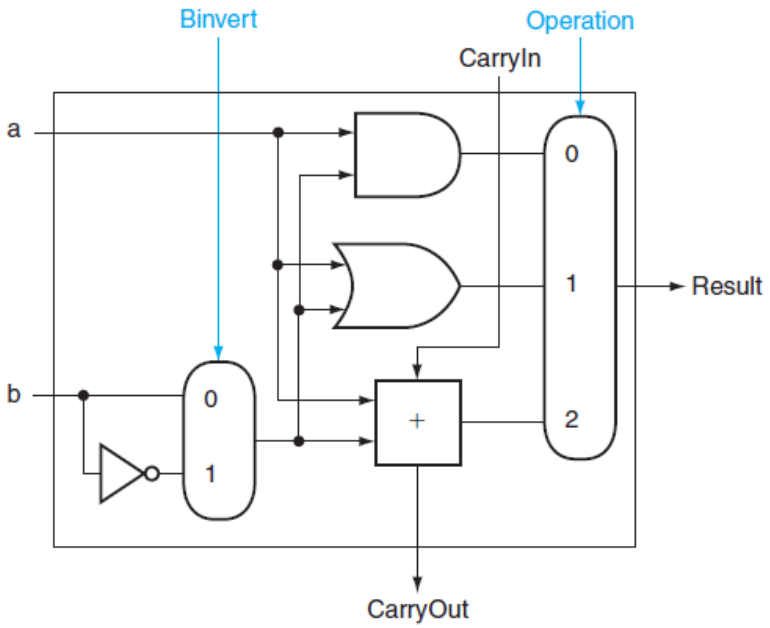
The 1-bit ALU that performs AND, OR, and addition.

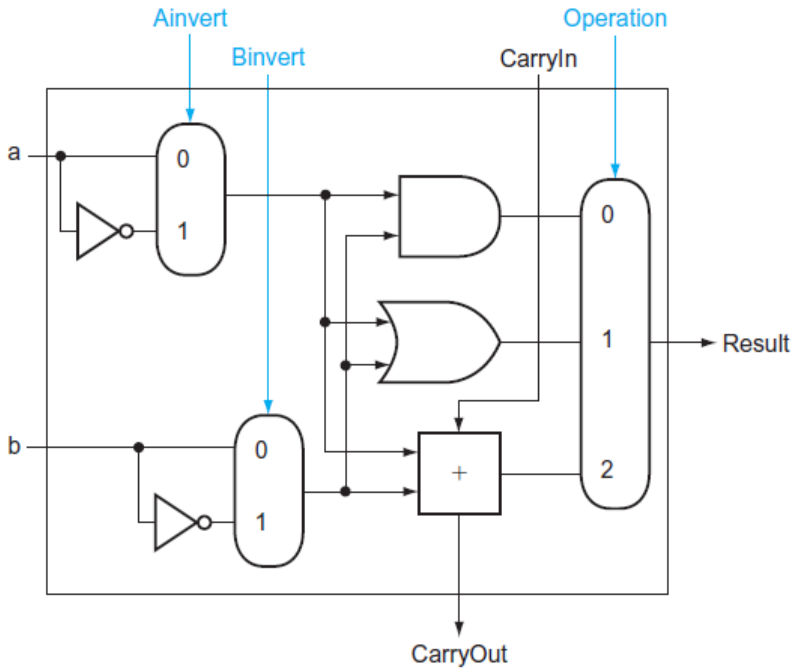


Support for subtraction (through addition).

- By selecting b (Binvert = 1) and setting CarryIn to 1 in the least significant bit of the ALU, we get two's complement subtraction of b from a instead of addition of b to a.

Arithmetic Logic Unit (ALU)





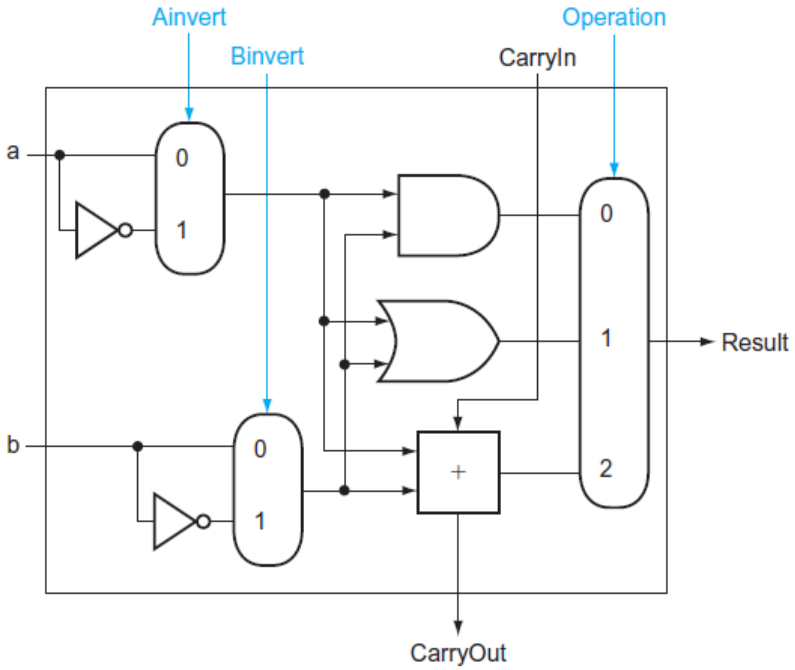
The 1-bit ALU that performs AND, OR, addition, and subtraction (through addition).

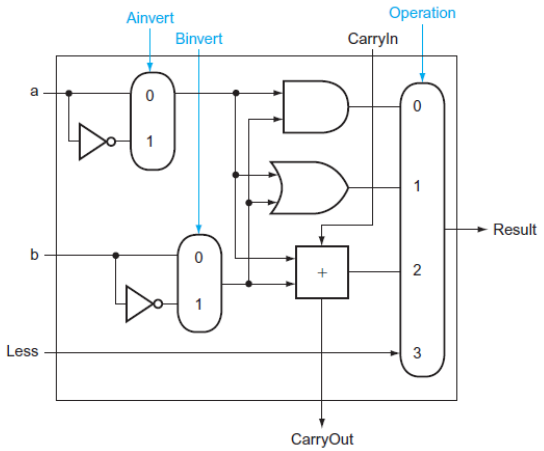
Support for NOR operation (through AND).

- DeMorgan's Theorem .

$$\overline{(a + b)} = \overline{a} \cdot \overline{b}$$

Arithmetic Logic Unit (AL



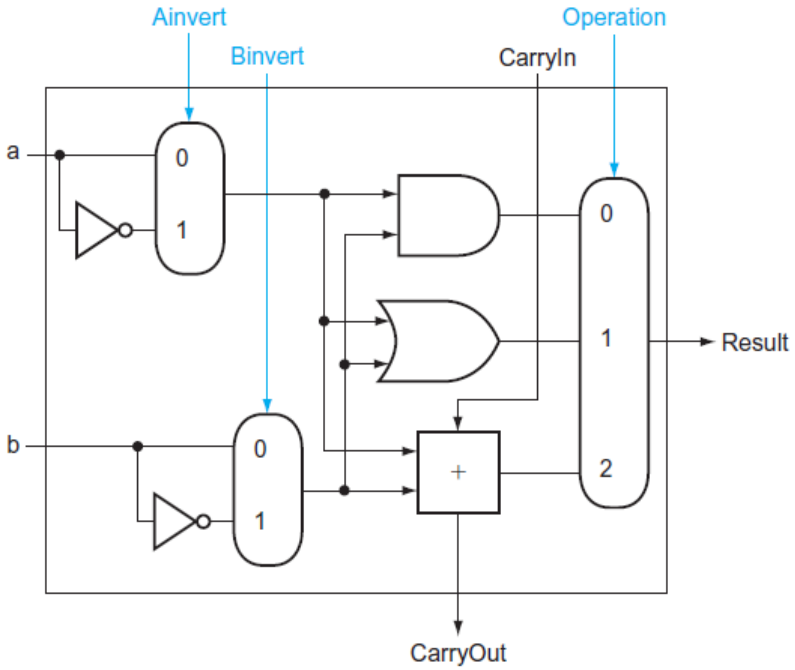


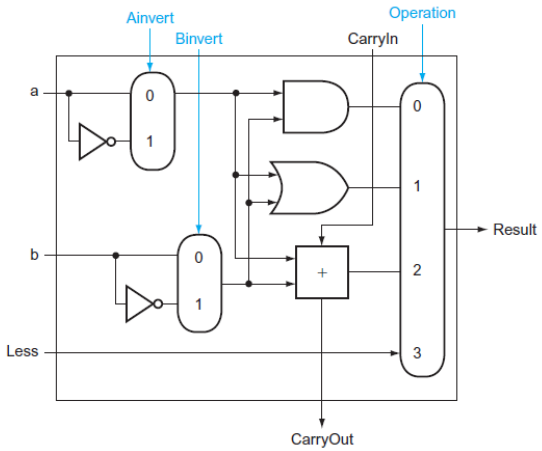
The 1-bit ALU that performs AND, OR, addition, subtraction (through addition) and NOR operation.

Support for slt operation.

- slt rd, rs1, rs2
  rd =1 if rs1<rs2
  rd = 0 otherwise
- Consequently, slt will set all but the least significant bit to 0, with the least significant bit set according to the comparison.

Arithmetic Logic Unit (AL



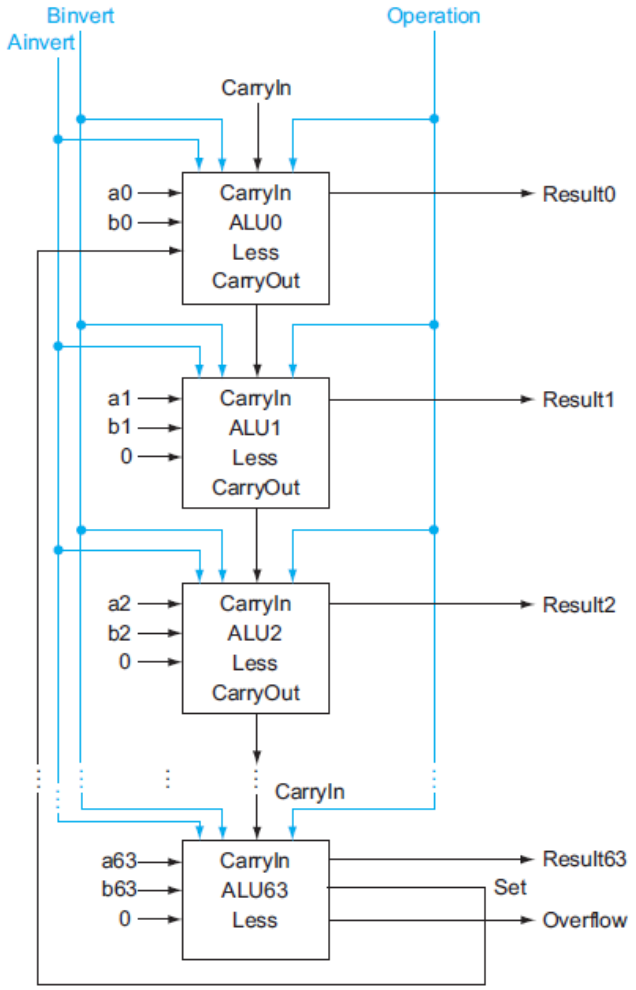


The 1-bit ALU that performs AND, OR, addition, subtraction (through addition) and NOR operation.

Support for slt operation.

- For the ALU to perform slt, we first need to expand the three-input multiplexor to add an input for the slt result. We call that new input Less and use it only for

Arithmetic Logic Unit (ALU)



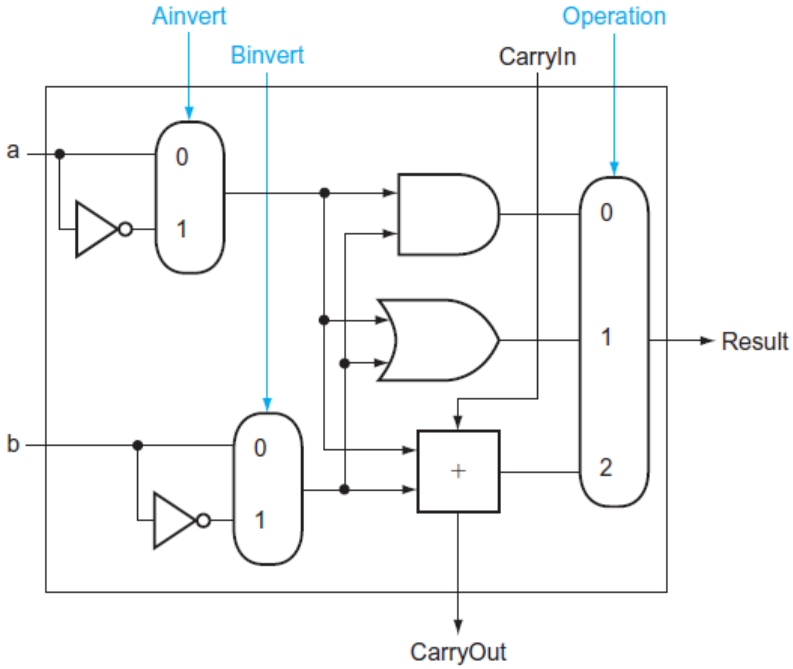
<u>Support for slt operation.</u>

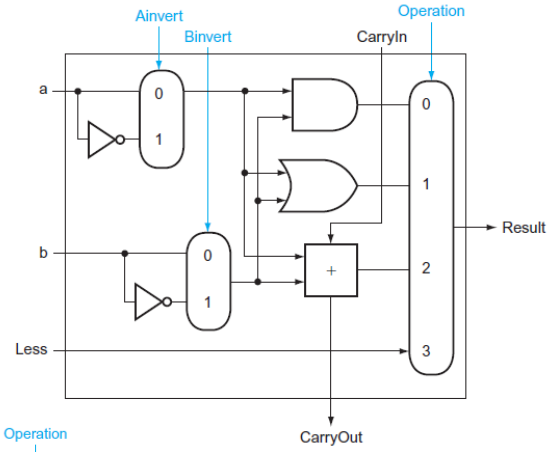
- We must connect 0 to the Less input for the upper 63 bits of the ALU, since those bits are always set to 0.

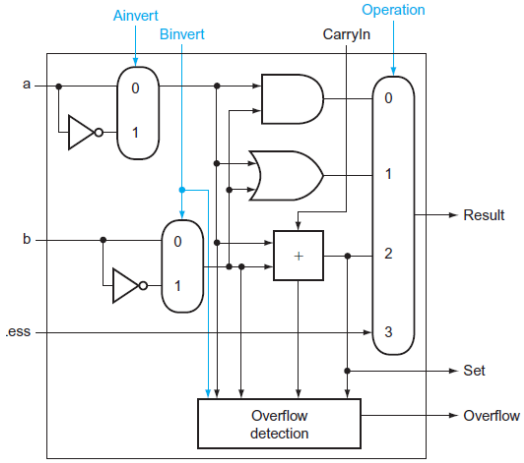
- How to set the least significant bit?
  - We want the least significant bit of slt operation to be a 1 if a < b
  - That is, we want the least significant bit of slt operation to be a 1 if a − b is negative and a 0 if a-b is positive
  - This desired result corresponds exactly to the sign bit values after performing a-b: 1 means negative and 0 means positive
  - So, we need only connect the sign bit (adder output of the most significant bit) to the Less input of least significant bit to get slt.
  - This needs a modified 1-bit ALU at the most significant bit

<u>A 64-bit ALU constructed from 64 1-bit ALUs (Notice the ALU for Most Significant Bit)</u>

Arithmetic Logic Unit (AL







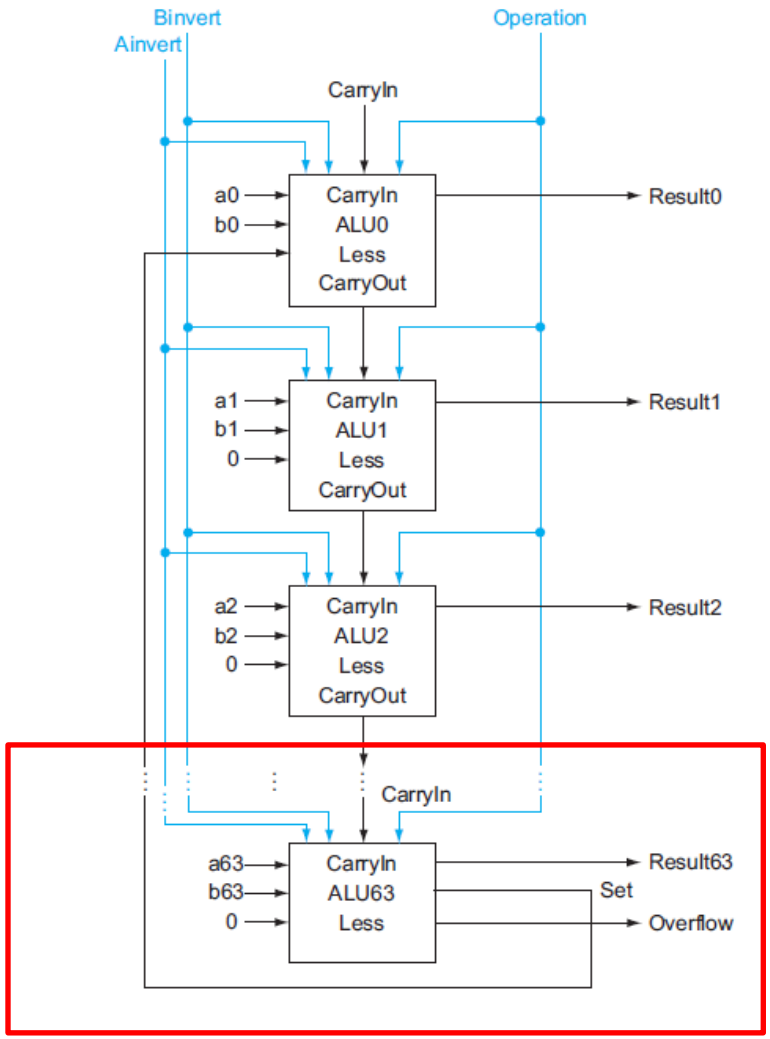
Most significant bit

The 1-bit ALU that performs AND, OR, addition, subtraction (through addition) and NOR operation.

Support for slt operation.

- We need to connect the sign bit (adder output of the most significant bit) to the Less input of least significant bit to get slt.
- This needs a modified 1-bit ALU at the most significant bit (addition of the Set output)

Arithmetic Logic Unit (ALU)



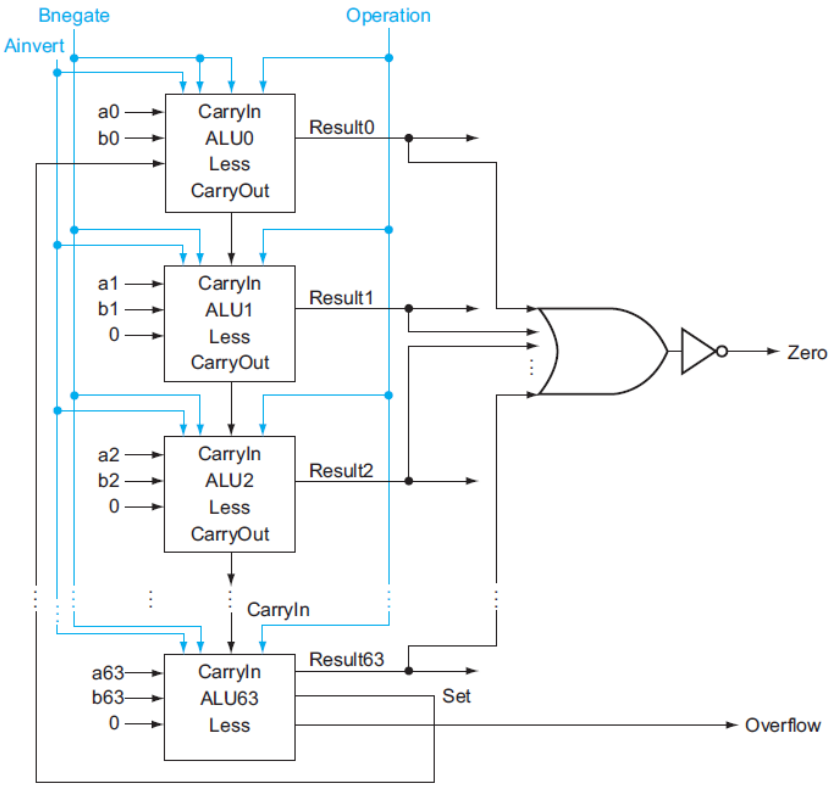
Support for slt operation.

- We must connect 0 to the Less input for the upper 63 bits of the ALU, since those bits are always set to 0.

- How to set the least significant bit?
  - We want the least significant bit of slt operation to be a 1 if a < b
  - That is, we want the least significant bit of slt operation to be a 1 if a − b is negative and a 0 if a-b is positive
  - This desired result corresponds exactly to the sign bit values after performing a-b: 1 means negative and 0 means positive
  - So, we need only connect the sign bit (adder output of the most significant bit) to the Less input of least significant bit to get slt.
  - This needs a modified 1-bit ALU at the most significant bit

A 64-bit ALU constructed from 64 1-bit ALUs (Notice the ALU for Most Significant Bit)

Arithmetic Logic Unit (ALU)

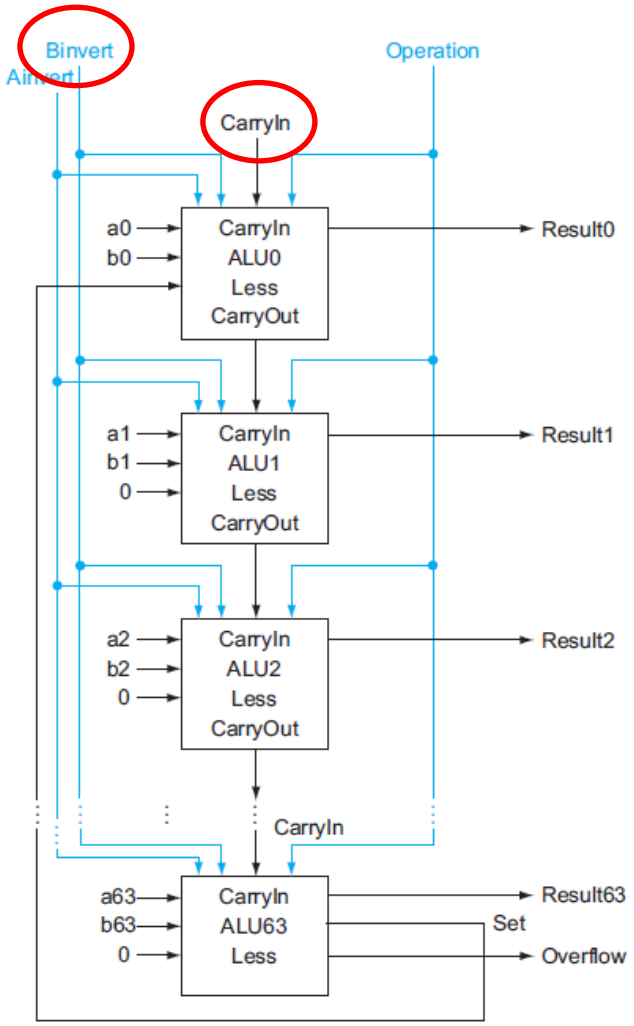


Support for beq operation.

- beq operation branches if two registers are equal.
- The easiest way to test equality with the ALU is to subtract b from a and then test to see if the result is 0.
- To test if the result is 0, the simplest way is to OR all the outputs together and then send that signal through an inverter

$$\text{Zero} = \overline{(\text{Result63} + \text{Result62} + \cdots + \text{Result2} + \text{Result1} + \text{Result0})}$$

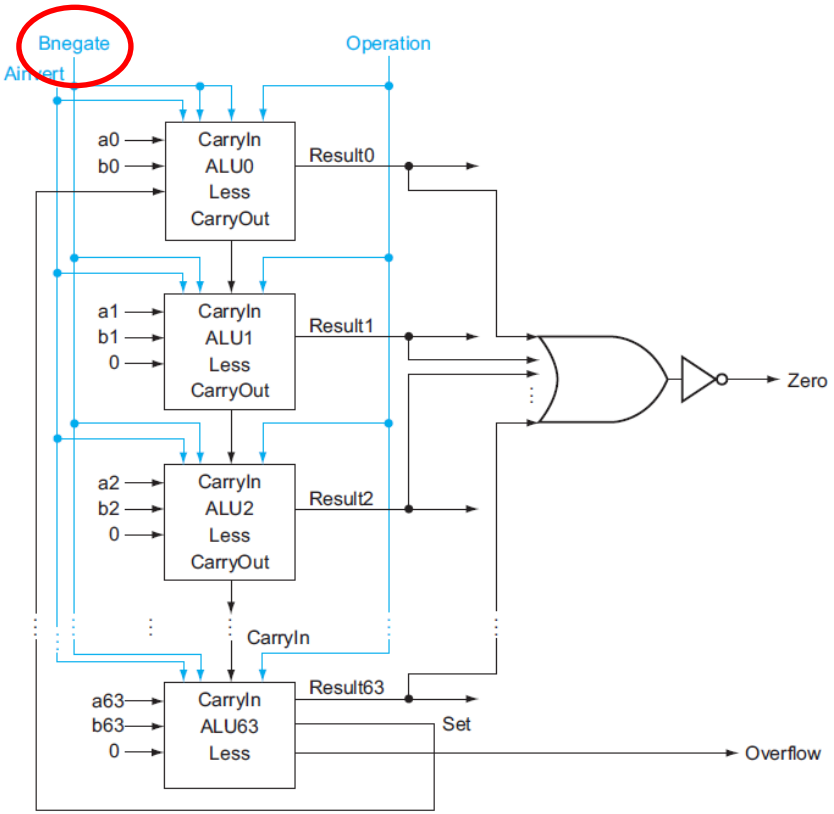
Arithmetic Logic Unit (ALU)



Simplifying Control

- Notice that every time we want the ALU to subtract, we set both CarryIn and Binvert to 1.
- For adds or logical operations, we want both control lines to be 0.
- We can therefore simplify control of the ALU by combining the CarryIn and Binvert to a single control line called Bnegate.

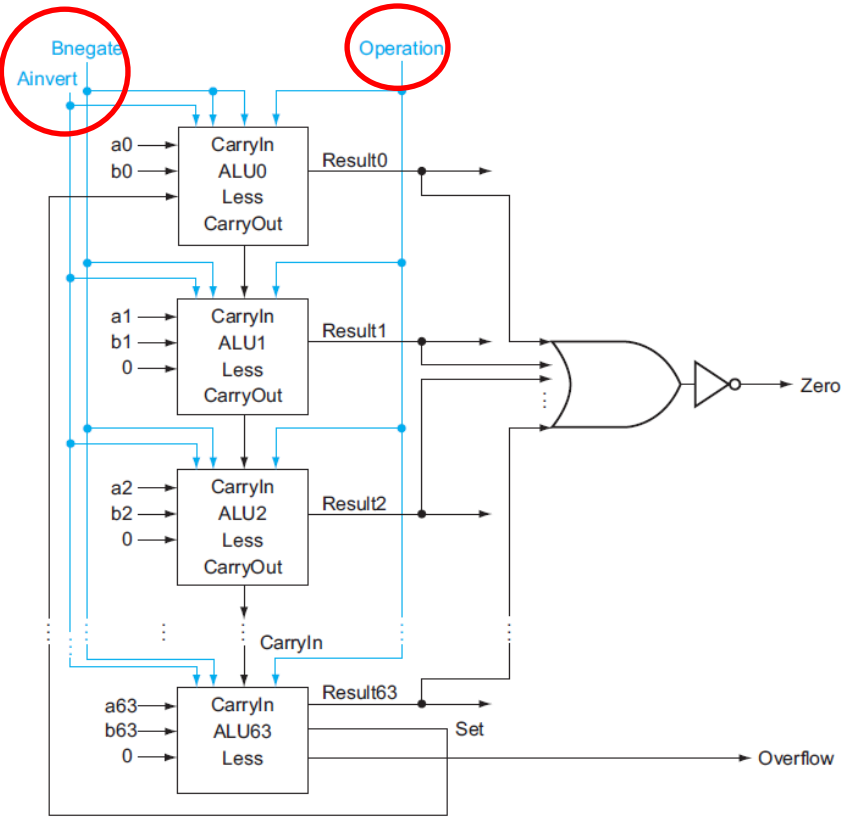
Arithmetic Logic Unit (ALU)



Simplifying Control

- Notice that every time we want the ALU to subtract, we set both CarryIn and Binvert to 1.
- For adds or logical operations, we want both control lines to be 0.
- We can therefore simplify control of the ALU by combining the CarryIn and Binvert to a single control line called Bnegate.

Arithmetic Logic Unit (ALU)

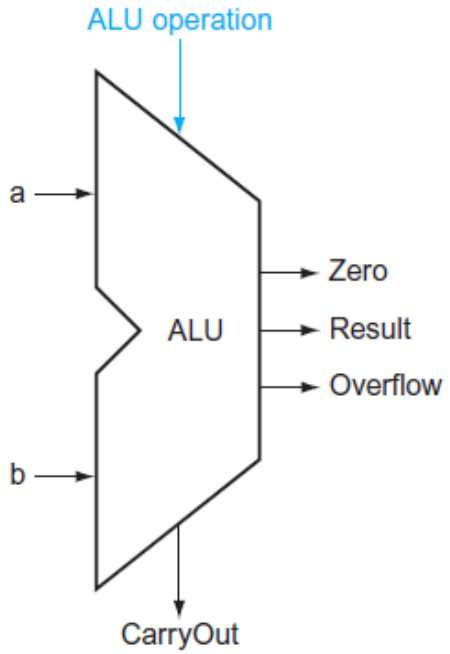


ALU Control

- We can think of the combination of the 1-bit Ainvert line, the 1-bit Bnegate line, and the 2-bit Operation lines as **4-bit control lines for the ALU**, telling it to perform add, subtract, AND, OR, NOR, o set less than.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR

Arithmetic Logic Unit (ALU)



Symbol for a Complete ALU